DiffusionNet

Pramook Khungurn

March 16, 2026

This article is written as I read the "DiffusionNet" paper by Sharp et al. [SACO22].

1 Introduction

- The goal of this paper is to design a neural network architecture for a data type that is not commonly used in mainstream deep learning: points sampled from a surface.
- There have been multiple attempts at this problem, but they are remaining problems.
  - Existing solutions are tied to a particular representation (e.g., triangle meshes or point clouds).
  - Mesh-based architectures do not scale well to high-resolution surface data. That is, performance degrades if the mesh is refined.
  - Existing approaches are too sensitive to underlying mesh structure.
- The paper proposes a solution that is both scalable and robust to sampling changes.
- Main observation.
  - Diffusion (i.e., evolution according to the heat equation) is highly effective for spatial communication.
  - Spatial gradient can be used to capture anisotropy.
- The code for this paper is available at `https://github.com/nmwsharp/diffusion-net`.

2 Method

- The method has three building blocks.
  1. Multi-layer perceptrons (MLPs) applied at each point independently to model point-wise scalar functions of feature channels.
  2. A learned diffusion operation for propagating information across the domain.
  3. Local spatial gradient features to expand the network's filter space beyond radially symmetric features.

2.1 Point-Wise MLPs

- As input we are given either a mesh or a point cloud with V vertices.
- Each vertex comes with a feature vector of dimension D.
- The MLP in question is a function $f : \mathbb{R}^D \to \mathbb{R}^D$, which is applied independently at every vertex to transform the feature.

- Because the MLP is applied independently to every point, it does not capture spatial structure of the surface.

2.2 Learned Diffusion

2.2.1 Background

- To review the Laplacian, see my notes [Khu25].
- Let $u(t, \mathbf{x})$ where $t \in \mathbb{R}$ and $\mathbf{x} = (x_1, x_2, \ldots, x_d) \in \mathbb{R}^d$ be a time-dependent scalar field on a $\mathbb{R}^d$.
- **Diffusion** is the process of evolving $u(t, \mathbf{x})$ according to the **heat equation**:

$$\frac{\partial u}{\partial t} = \Delta u$$

  where Δu is the **Laplacian** of u; that is,

$$\Delta u = \sum_{i=1}^{d} \frac{\partial^2 u}{\partial x_i^2}.$$

- In particular, $\Delta u(t, \mathbf{x})$ indicates the difference between the value $u(t, \mathbf{x})$ and the average value of u in the neighborhood of $\mathbf{x}$.
  - If $\Delta u(t, \mathbf{x}) > 0$, then $u(t, \mathbf{x})$ is lower than the average value of u in the neighborhood of $\mathbf{x}$.
  - If $\Delta u(t, \mathbf{x}) < 0$, then $u(t, \mathbf{x})$ is greater than the average value of u in the neighborhood of $\mathbf{x}$.
  - If $\Delta u(t, \mathbf{x}) = 0$, then $u(t, \mathbf{x})$ is equal to the average value of u in the neighborhood of $\mathbf{x}$.
- So, evolution according to the heat equation is a smoothing process.
  - If $\Delta u(t, \mathbf{x}) > 0$, then $u(t, \mathbf{x})$ gets increased.
  - If $\Delta u(t, \mathbf{x}) < 0$, then $u(t, \mathbf{x})$ gets decreased.
  - If $\Delta u(t, \mathbf{x}) = 0$, then $u(t, \mathbf{x})$ is unchanged.

  If we let $u(t, \mathbf{x})$ evolve over time, peaks will be flattened, and depressions will be filled.
  u will tend towards a constant function, whose value is the average of u over the domain.
- We call the process "diffusion" because information from various parts of the domain must travel to other parts in order for the system to be able to compute the average.
- The paper says that using the heat equation ensures that the results are largely invariant to the way the surface is sampled or meshed.
  - This is because propagation depends only on the domain size and the passage of time, not on how the domain is sampled.

2.2.2 Discretization of the Laplacian

- To perform computation with a computer, one must replace Δ with matrices.
- There are two matrices involved.
  - The **weak Laplace matrix** $L \in \mathbb{R}^{V \times V}$ is positive semi-definite and sparse.
  - The **mass matrix** $M \in \mathbb{R}^{V \times V}$ is a diagonal matrix.

- The paper follows the convention that the signs of the mass matrix should be such that

$$M^{-1}L \approx -\Delta.$$

- On triangle meshes, we can use the cotan-Laplace matrix [Cra25].
  - Let there be T triangles in the mesh.
  - The **gradient operator** ∇ is matrix of size $3T \times V$ that converts a scalar field on the vertices (represented by a $V \times 1$ vector) to a vector field on the triangles (represented by a $3T \times 1$ vector).
    - `libigl` library provides a function `grad` that computes this matrix.
  - Area matrix $\mathcal{A}$ is a $3M \times 3M$ diagonal matrix whose diagonal entries are the areas of the triangles multiplied by two.
    - `libigl` library provides a function `doublearea` that computes the area of each triangle, times two.
  - The cotan-Laplace matrix is given by:

$$L = \Delta = \nabla^T \mathcal{A} \nabla.$$

  - I'm not so sure how to define the mass matrix M in this case. Is it just $-I$?
- For point clouds, the paper says it uses the Laplacian from the paper by Sharp and Crane [SC20].
- Sharp and Crane provides a library for computing L and M for both triangle meshes and point clouds.
  - `https://github.com/nmwsharp/robust-laplacians-py`
  - If you use the `diffusion-net` library, make sure to have the above library as a dependency.
- The rate of diffusion Δu is given by

$$\Delta u = -M^{-1}Lu.$$

2.2.3 Learned Diffusion Layer

- A learned diffusion layer h_t is a function of signature $h_t : \mathbb{R}^V \to \mathbb{R}^V$.
  - The layer diffuses feature u for learned time $t \geq 0$.
  - The layer is applied independently to each feature channel, with a separate learned time t per-channel.
- Learning the time is the strength of the method.
  - It allows the network to continuously optimize for spatial support, ranging from purely local to totally global.
  - This sidesteps the manual choosing of the support radius of a convolution or the size of the pooling hierarchy.

2.2.4 Computing Diffusion

- We need a scheme to compute diffusion that is both efficient and differentiable.
- The paper considers two schemes.
  - **Direct implicit timestep.**
    - We simulate diffusion with a single implicit Euler timestep.
    - That is, we start from $u(t)$, which is unknown, and we go backward in time to $u(0)$ with one single Euler integration step:

      $$u(0) \approx u(t) - t\Delta u(t)$$

      As a result,

      $$\begin{aligned} u(0) &\approx u(t) + tM^{-1}Lu(t) \\ u(0) &\approx (I + tM^{-1}L)u(t) \\ Mu(0) &\approx (M + tL)u(t) \\ u(t) &\approx (M + tL)^{-1}Mu(0). \end{aligned}$$

      In other words,

      $$h_t(u) = (M + tL)^{-1}Mu.$$

    - This involves solving a (sparse) linear system for each diffusion operation.
    - Using an implicit backward timestep rather than an explicit forward timestep is crucial.
      - It makes the scheme stable.
      - It allows global support.
      - It yields a reasonable approximation of diffusion after just one step.
  - **Spectral acceleration.**
    - There is a closed-form expression for diffusion in the basis of low-frequency Laplacian eigenfunctions [VL07, LZ10].
    - Once the eigenbasis has been precomputed, diffusion can be evaluated for any time t via element-wise exponentiation.
    - Truncating diffusion to a low-frequency basis incurs some approximation error, but the authors found it had little effect on the final performance. (Perhaps because diffusion quickly damps high-frequency components regardless.)
    - For weak Laplace matrix L and mass matrix M, the eigenvectors $\phi_i \in \mathbb{R}^V$ are solutions to:

      $$L\phi_i = \lambda_i M\phi_i$$

      where λ_i is a scalar eigenvalue.
    - We collect the eigenvectors for the first k smallest-magnitude eigenvalues $\lambda_1, \lambda_2, \ldots, \lambda_k$.
    - We normalize the eigenvectors so that $\phi_i^T M\phi_i = 1$.
    - We can compute the eigenvectors with standard numerical method packages.
    - Let $\Phi = \begin{bmatrix}\phi_1 & \phi_2 & \cdots & \phi_k\end{bmatrix} \in \mathbb{R}^{V\times k}$ be stacked matrix of eigenvectors.
      - It forms an orthonormal basis with respect to M.
    - We can project any scalar function u to obtain its coefficients c in the spectral basis via $c = \Phi^T Mu$.

    * Diffusion for time t for the coefficient c_i is given by

$$c_i(t) = e^{-\lambda_i t} c_i(0).$$

    * As a result, we have that

$$h_t(u) = \Phi \begin{bmatrix} e^{-\lambda_1 t} \\ e^{-\lambda_2 t} \\ \vdots \\ e^{-\lambda_k t} \end{bmatrix} \odot (\Phi^T M u).$$

    Here, $\odot$ denotes element-wise multiplication.

- Another way to do this is to perform multi-step forward Euler steps and get the gradient back through the use of adjoint method, as pointed out in the neural ODE paper [CRBD18].

2.2.5 Spatial Gradient Features

- The learned diffusion layer enables propagation of information across different points on a shape, but only supports radially symmetric filters about a point.
- The last building block enables a larger space of filters by computing additional features from the spatial gradients of signal values at vertices.
- We construct features from the inner products between pairs of feature gradients at each vertex, after applying a learned scaling or rotation.
- The spatial gradient of a scalar function on a surface is represented as a 2D vector in the tangent space of each vertex.
- These gradients can be evaluated by a standard procedure.
  - Choose a normal vector at each vertex.
  - Project neighbors into the tangent plane.
    * 1-ring neighbor for a mesh.
    * m-nearest neighbors for a point cloud.
  - The gradient is then computed in the tangent plane via least-squares approximation of the function values at neighboring points [MW06].
  - These gradient operators at each vertex can assembled into a sparse matrix $G \in \mathbb{C}^{V \times V}$, which is applied to a vertex u for real values at vertices to produce gradient tangent vectors at each vertex.
    * The matrix does not depend on the features.
    * It can be precomputed once for each shape.
    * The complex vector is used as a convenient notation for 2D vectors.
- Equipped with per-vertex spatial gradients for each channel, we learn informative gradient features by evaluating inner product between pairs of feature gradients at each vertex, after a learned linear transformation.
- Inner products are invariant to rotations of the coordinate systems, so these features are invariant to the choice of tangent basis at vertices.
- The computation of the gradient features is as follows.

- For each channel u in the D scalar feature channels, we construct its spatial gradient as $z_u \in \mathbb{C}^V$, which is given by

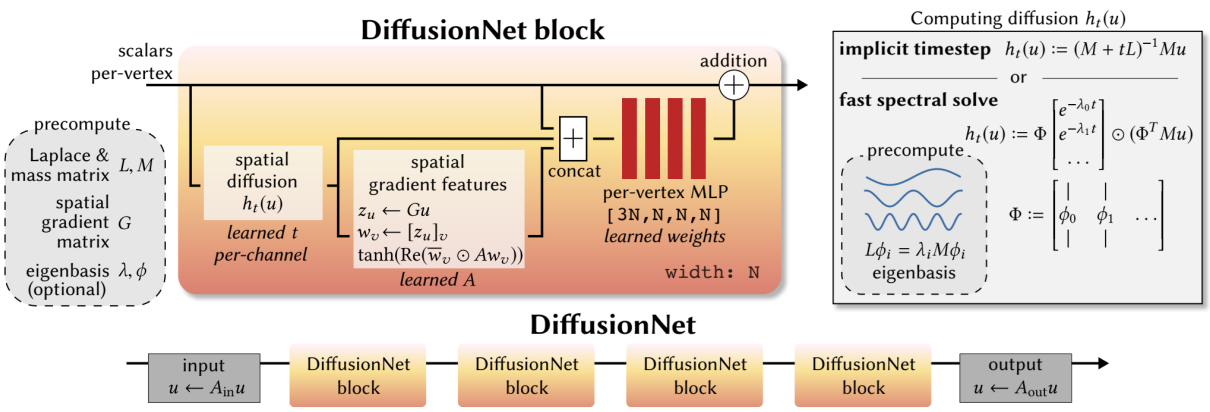
$$z_u = Gu$$

  for some matrix $G \in (2V) \times V$.
- At each vertex v, we stack the local gradients of all channels to form $w_v \in \mathcal{C}^D$.
- The feature $g_v \in \mathbb{R}^D$ is obtained as

$$g_v = \tanh\left(\Re(\overline{w_v} \odot Aw_v)\right).$$

  Here:
  * A is a learned square $D \times D$ matrix.
  * Taking the real part after a complex conjugate $\overline{w_v}$ is just a notation convenience for dot products between pairs of 2D vectors.
  * The $\tanh(\cdot)$ is not fundamental, but the paper says that it stabilizes the training.

3 DiffusionNet Architecture



- A **DiffusionNet architecture** is composed of several **DiffusionNet blocks**.
- A DiffusionNet block starts with diffusion of input features, computation of spatial gradient features and then followed by per-point feature transformation with an MLP. The result is then added to the incoming features like what is done in ResNet.
- There is no need for spatial convolution or pooling on surfaces.

References

[Cra25] K. Crane, *Discrete Differential Geometery: An Applied Introduction*, `https://www.cs.cmu.edu/~kmcrane/Projects/DDG/paper.pdf`, 2025, Accessed: 2025-04-16.

[CRBD18] R. T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. K. Duvenaud, *Neural ordinary differential equations*, Advances in Neural Information Processing Systems (S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett, eds.), vol. 31, Curran Associates, Inc., 2018.

[Khu25] P. Khungurn, *Laplacians (aka laplace operators)*, `https://pkhungurn.github.io/notes/notes/math/laplacians/index.html`, 2025.

[LZ10] B. Lévy and H. R. Zhang, *Spectral mesh processing*, ACM SIGGRAPH 2010 Courses (New York, NY, USA), SIGGRAPH '10, Association for Computing Machinery, 2010.

[MW06] S. Mukherjee and Q. Wu, *Estimation of gradients and coordinate covariation in classification*, J. Mach. Learn. Res. **7** (2006), 2481–2514.

[SACO22] N. Sharp, S. Attaiki, K. Crane, and M. Ovsjanikov, *Diffusionnet: Discretization agnostic learning on surfaces*, ACM Trans. Graph. **41** (2022), no. 3.

[SC20] N. Sharp and K. Crane, *A Laplacian for Nonmanifold Triangle Meshes*, Computer Graphics Forum (SGP) **39** (2020), no. 5.

[VL07] B. Vallet and B. Lévy, *Spectral Geometry Processing with Manifold Harmonics*, Technical report, 2007.